# Low-Level Design Report

## 1. Introduction

This document presents the Low-Level Design (LLD) details of the "Manifetch" web-based clinical decision support platform designed for Neonatal Intensive Care Units (NICU). It represents the stage where the system design is elaborated to serve as a basis for implementation, and where architectural decisions are concretized at the level of classes, function signatures, and data structures.

In the previous High-Level Design (HLD) phase, what the system would do was defined, and the overall boundaries of modules such as Data Ingestion, AI Inference, Backend, and Frontend were outlined. This LLD report, on the other hand, focuses on how these functions will be carried out technically. The main modules defined in the HLD are decomposed in this document into classes, function signatures, and data structures.

The Manifetch project has strict performance constraints, such as processing real-time physiological signals (ECG, SpO2) and immediately alerting healthcare staff when predefined clinical thresholds are exceeded (maximum 1-second latency). Therefore, throughout this document, core workflows—such as transferring sensor data from the simulation layer to the dashboard, executing the rule engines, and triggering alarms [8] in anomaly cases followed by staff acknowledgement—are addressed with a lean and implementable object-oriented approach.

This document will serve as a reference during implementation for the development team and will clearly present the system's internal operation to the academic jury. All structural decisions throughout the design have been shaped in line with data privacy principles, under the assumption that the system will operate only with simulated data and within a single hospital.

### 1.1 Object design trade-offs

In line with the system's real-time monitoring requirements, the following design trade-offs have been made:

- **Low Latency vs. Model Complexity:** The system is constrained to a maximum latency of 1 second for live signal streaming and 3 seconds for AI inference. Therefore, instead of overly complex deep learning models that could significantly increase processing time, the design favors decision-support approaches that can generate real-time alerts, maintain clinically acceptable accuracy, and be optimized to meet the latency targets.

- **Security vs. Usability:** To protect medical data, the system applies Role-Based Access Control and considers security requirements such as TLS 1.3 [5] for data transmission and AES-256 [6] for data storage. To reduce unauthorized access, additional measures have been introduced, which also restricts usability.

- **Modularity vs. Performance Overhead:** Dividing the system into separate components—Frontend, Backend, AI Module, and Data Simulation—introduces additional overhead in inter-component communication. However, this approach is preferred over a monolithic structure due to improved independent testability, easier maintenance, and clearer separation of development responsibilities [9].

### 1.2 Interface documentation guidelines

In this document, standard programming practices are followed for naming classes, methods, and attributes. Class names are written in PascalCase (e.g., *VitalMeasurement*), while method and attribute names use camelCase (e.g., *getPatientDetails*). When defining class interfaces, the class's role within the system is specified, along with its attributes including data types, and its methods including their input parameters and return values.

### 1.3 Engineering standards

- **UML [3] (Unified Modeling Language):** Used as the standard for visualizing class and component diagrams.

- **IEEE Documentation Practices:** The report structure and requirements traceability are organized in line with IEEE [2] oriented documentation approaches.

- **HL7/FHIR Principles:** Although the project will use simulated data, the logical hierarchy of data objects (patient, measurement, alarm) is designed with reference to FHIR [1] data models, which are a widely used healthcare data standard.

### 1.4 Definitions, acronyms, and abbreviations

- **AI (Artificial Intelligence):** An AI-based decision support and anomaly detection component.

- **ECG (Electrocardiogram):** A physiological signal that measures the electrical activity of the heart.

- **HLD / LLD:** High-Level Design / Low-Level Design.

- **NICU:** Neonatal Intensive Care Unit.
- **RBAC:** Role-Based Access Control.
- **SpO2:** Peripheral blood oxygen saturation

## 2. Packages

The project is organized into four main packages according to responsibility separation and real-time operation needs: Backend, Frontend, AI Module, and Data Simulation. This decomposition keeps clinical workflows (monitoring, rule evaluation, alerting, acknowledgement) traceable across the system while allowing independent development and testing of each layer [11]. Communication between packages is performed via REST for standard operations and WebSocket [4] for real-time streaming.

### 2.0 Package Interaction Overview

- **Data Simulation → Backend:** Streams synthetic physiological signals (ECG, SpO2) over WebSocket.
- **Backend → Frontend:** Exposes REST endpoints for authentication [7] and CRUD operations, and WebSocket channels for real-time vitals and alerts.
- **Backend ↔ AI Module:** Sends measurement windows (features) for inference and receives AIResult outputs (risk score, label, timestamp), typically via REST (or internal calls if co-deployed).
- **Frontend → Backend:** Sends alert acknowledgements (ACK), configuration changes, and user actions via REST/WebSocket.

### 2.1 Backend Package (Server)

The Backend package executes the core business logic of the system. It is responsible for managing patient context, receiving and validating measurement data, evaluating patient-specific threshold rules, handling the alert lifecycle (create–acknowledge–resolve), and serving data to the dashboard.

#### 2.1.1 Model Package

This subpackage contains the core domain classes and data structures of the system [10].

Patient information, measurements, threshold rules, alert records, and AI outputs are represented at this layer. Relationships between model classes are designed to support requirements such as patient-specific threshold management and traceable alert generation linked to both the triggering rule and measurement.

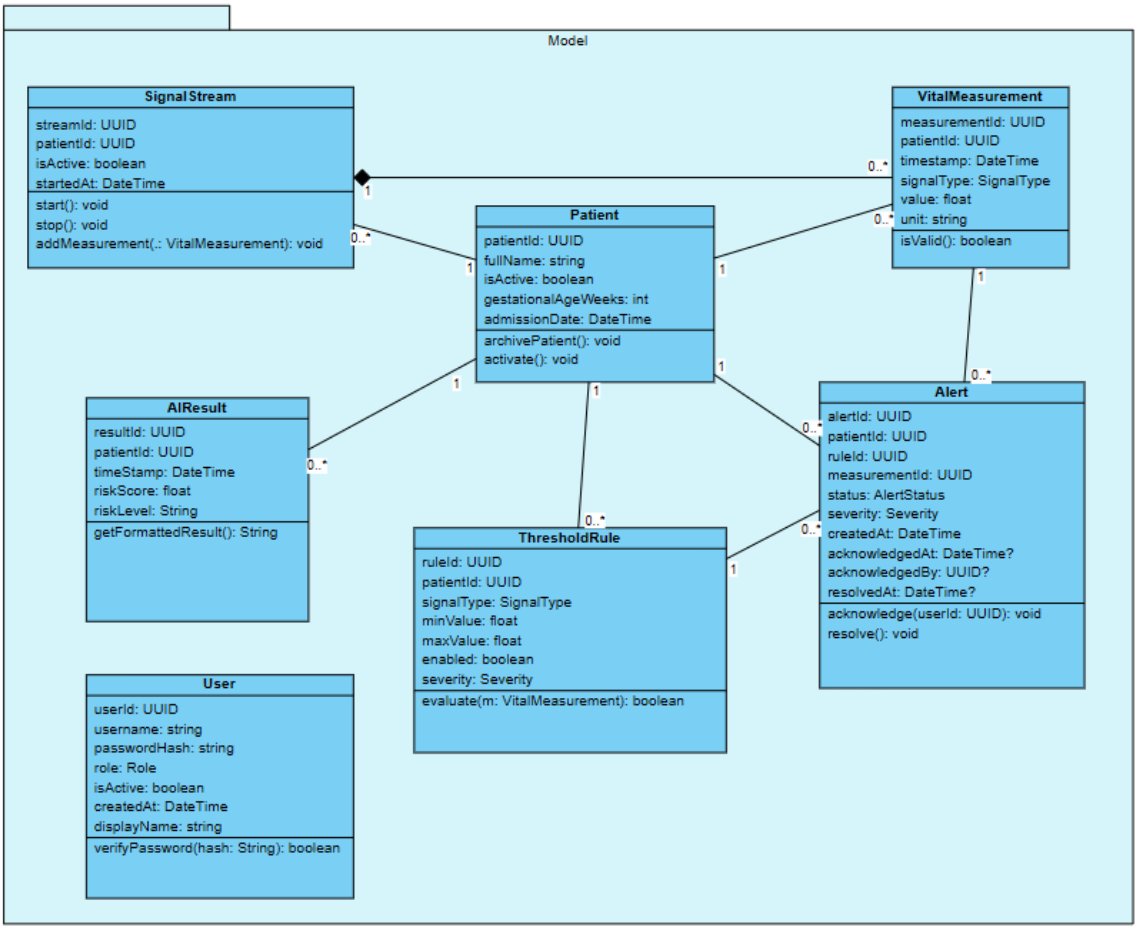


Figure 1: Backend / Model Class Diagram

### 2.1.2 Controller & Service Package

This subpackage includes controllers that handle requests coming from the client (Frontend) and the simulation layer, as well as service classes that execute business logic. Dashboard-related needs (patient details and active alerts) are provided via controllers. Measurements arriving from the simulation/streaming layer are received by the ingestion controller and routed to AlertService to initiate the evaluation flow. Threshold checks are performed through RuleEngineService based on patient-specific rules.

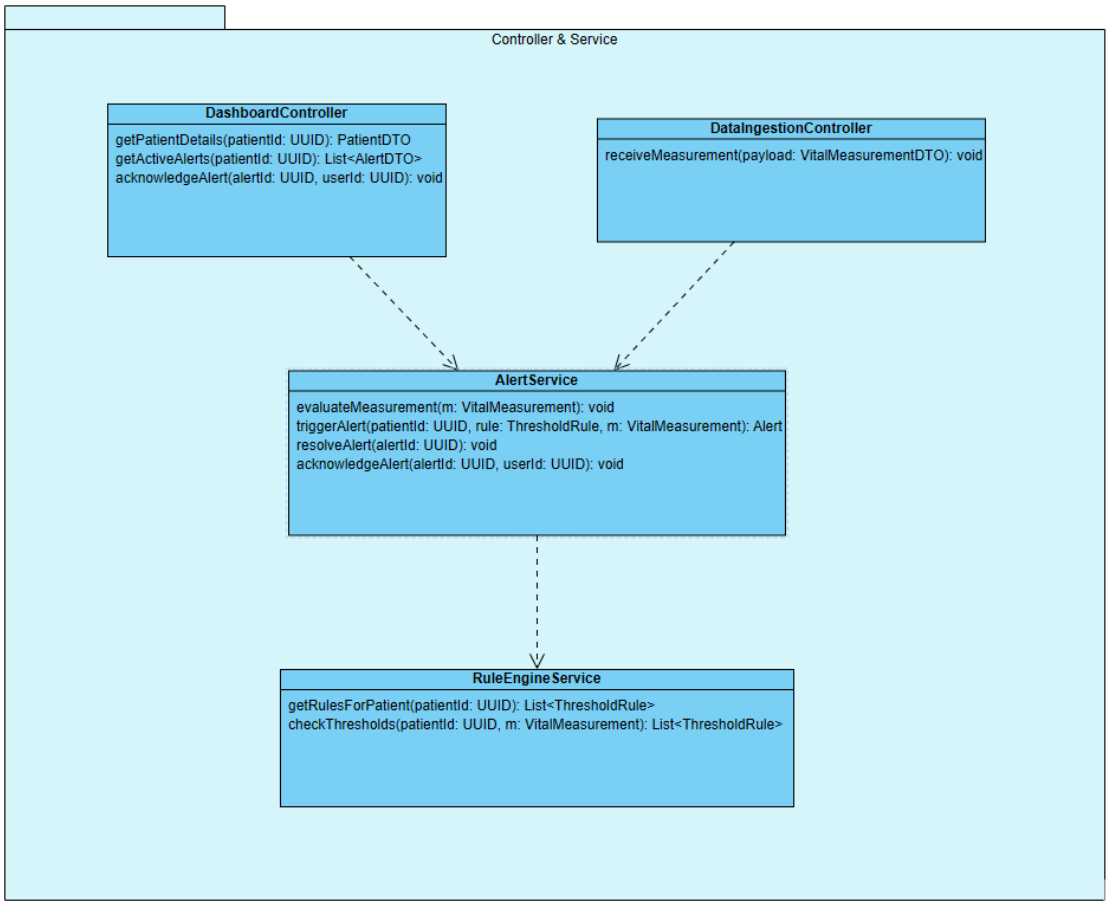


Figure 2: Backend / Controller & Service Diagram

### 2.2 Frontend Package (Client)

The Frontend package provides the NICU monitoring interface (dashboard) to users. It visualizes real-time measurements, displays active alerts, and forwards user actions (e.g., alert acknowledgement) to the backend. Communication between the frontend and backend is handled via REST for standard data retrieval (such as patient details) and WebSocket for live vitals and alert notifications.

#### 2.2.1 View & Network Package

View components build the user interface (dashboard screens). Network components manage communication with the backend (REST + WebSocket). DashboardPage

receives real-time messages through WebSocketClient to update charts and trigger alarm UI components, while standard data fetching is performed via ApiClient.

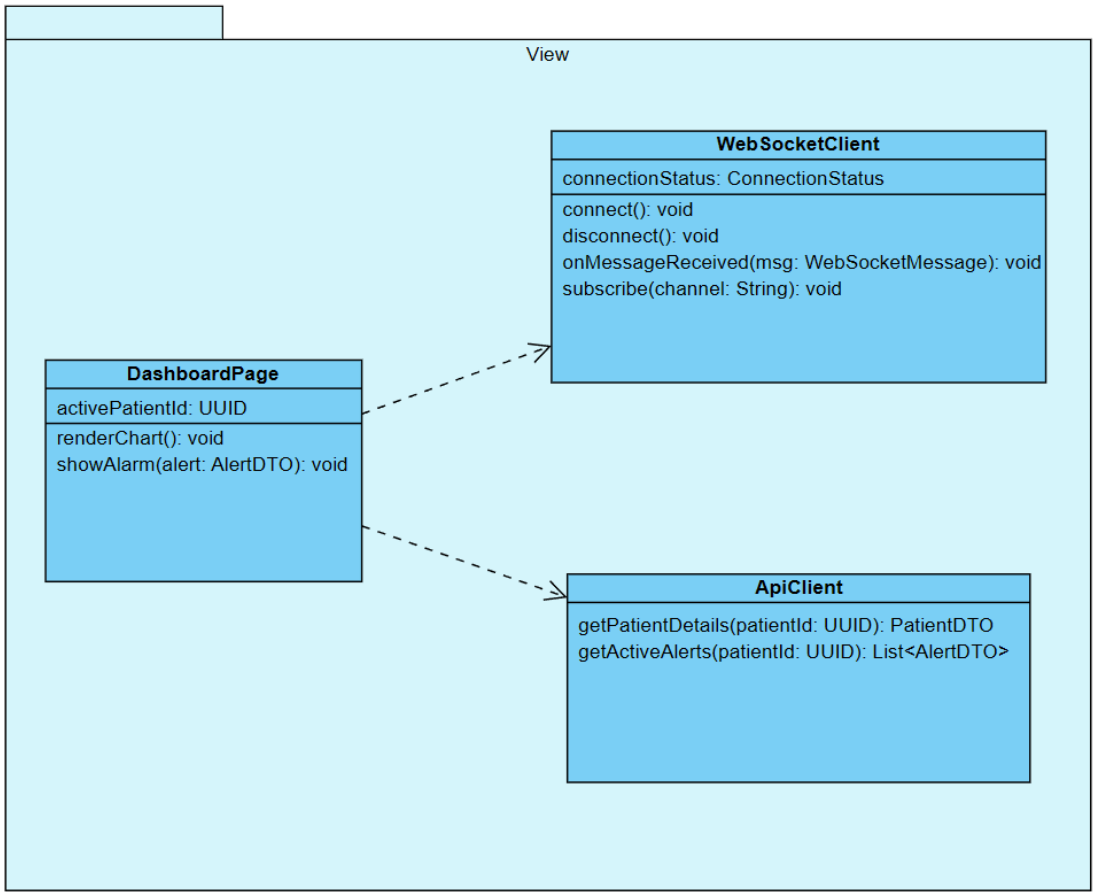


Figure 3: Frontend / View & Network Diagram

### 2.3 AI Module Package

The AI Module is an independent inference component that produces a risk score from incoming measurement data to provide decision-support output. Its purpose is to prepare inputs for the model (preprocessing), execute inference (prediction), and return results to the backend in a standardized output format (AIResult). This enables the backend to use risk evaluation in alerting and dashboard presentation workflows.

#### 2.3.1 Inference Subpackage

InferenceController serves as the entry point for inference requests. InferenceService orchestrates the end-to-end inference flow: it preprocesses input measurements into a

model-ready representation and obtains predictions via ModelRunner. The output is constructed as an AIResult and returned to the upper layer.

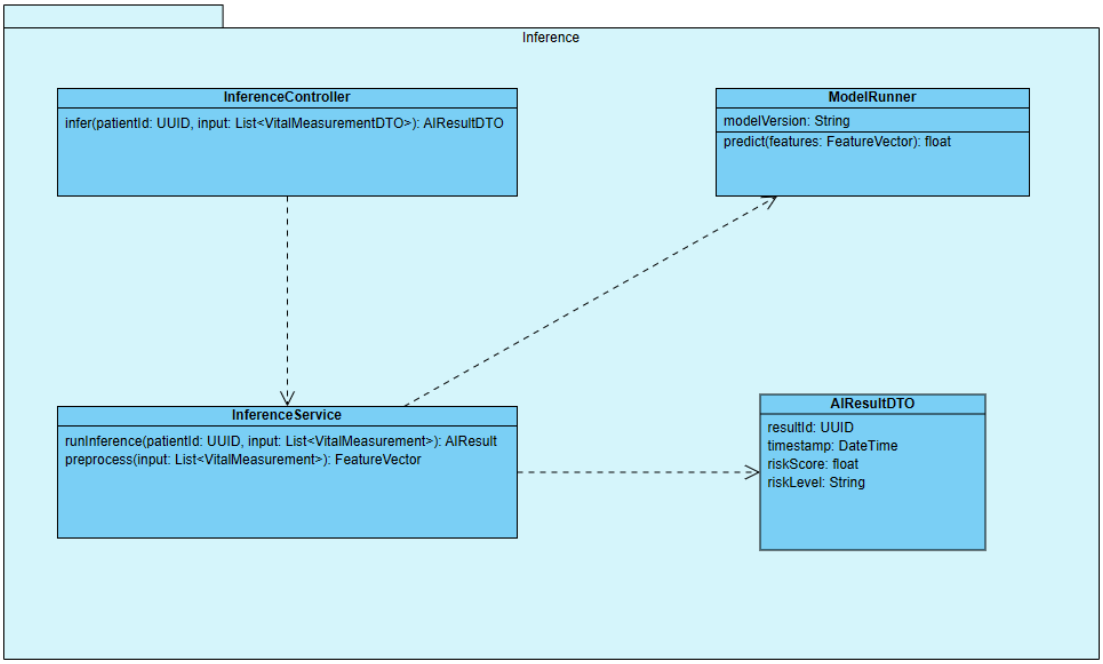


Figure 4: AI Module / Inference Diagram

### 2.4 Data Simulation Package

The Data Simulation package generates and transmits synthetic physiological measurements to emulate NICU monitors. Under the assumption that no real patient data is used, this package provides the necessary streams for testing and demo scenarios. Generation behavior is parameterized via ScenarioConfig, and produced measurements are sent to the backend through StreamPublisher.

#### 2.4.1 Simulation Subpackage

Simulator generates synthetic vital measurements. ScenarioConfig contains simulation parameters such as patient identifier, sampling rate, duration, and signal types. StreamPublisher establishes the connection and publishes generated measurements to the configured streaming channel.

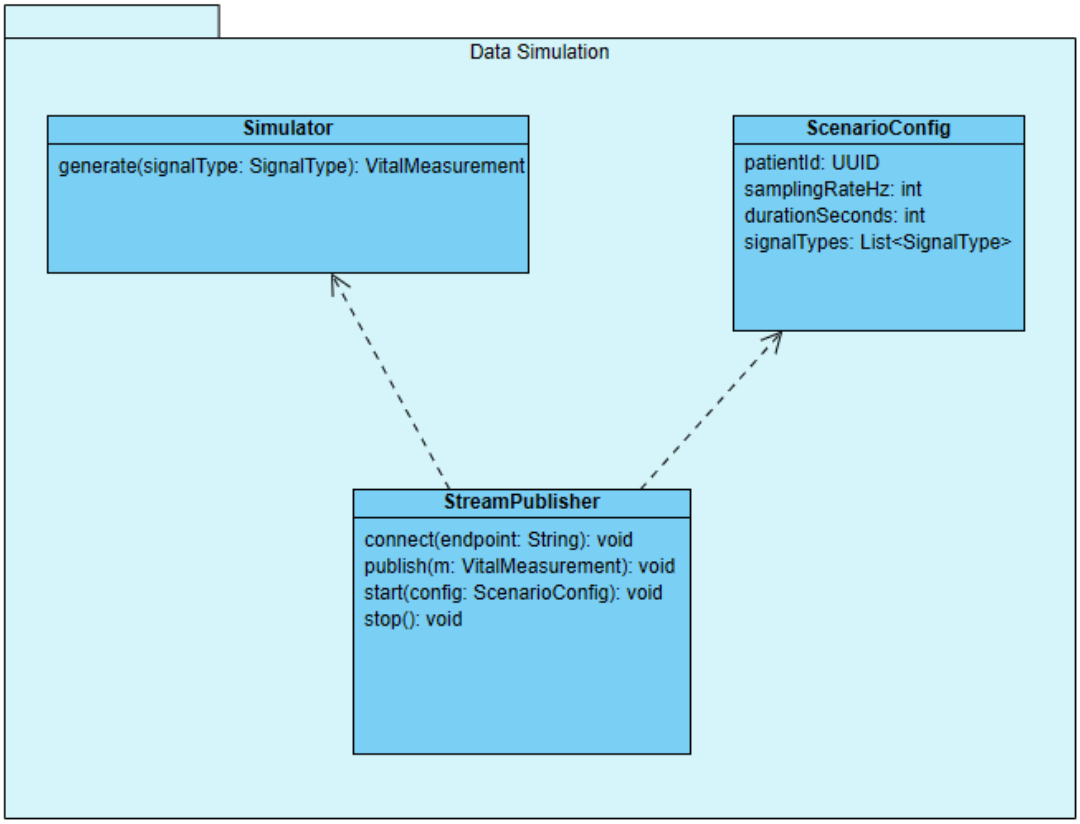


Figure 5: Data Simulation Diagram

## 3. Class Interfaces

### 3.1 Backend

#### 3.1.1 Model

**Class 1:** Patient

Patient represents the monitored neonate context in the NICU. Measurements, threshold rules, alerts, and AI outputs are associated through the patient entity.

**Attributes**

| | |
|---|---|
| patientId | UUID |
| fullName | string |
| isActive | boolean |

| | |
|---|---|
| gestationalAgeWeeks | int |
| admissionDate | DateTime |

**Methods**

| |
|---|
| archivePatient(): void |
| activate(): void |

**Class 2:** SignalStream

SignalStream represents a live signal channel for a specific patient and manages the measurement streaming lifecycle.

**Attributes**

| | |
|---|---|
| streamId | UUID |
| patientId | UUID |
| isActive | boolean |
| startedAt | DateTime |

**Methods**

| |
|---|
| start(): void |
| stop(): void |
| addMeasurement(m: VitalMeasurement): void |

**Class 3:** VitalMeasurement

VitalMeasurement is a data object that represents a single timestamped physiological measurement.

**Attributes**

| | |
|---|---|
| measurementId | UUID |
| patientId | UUID |
| timestamp | DateTime |
| signalType | SignalType |
| value | float |
| unit | string |

**Methods**

| |
|---|
| |
| isValid(): boolean |
| |

**Class 4:** ThresholdRule

ThresholdRule defines patient-specific threshold bounds and evaluates whether an incoming measurement violates the configured limits.

**Attributes**

| | |
|---|---|
| ruleId | UUID |
| patientId | UUID |
| signalType | SignalType |
| minValue | float |
| maxValue | float |
| enabled | boolean |
| severity | Severity |

**Methods**

| |
|---|
| |
| evaluate(m: VitalMeasurement): boolean |
| |

**Class 5:** Alert

Alert represents an alarm record triggered by a threshold violation and captures the alert lifecycle (active/acknowledged/resolved).

**Attributes**

| | |
|---|---|
| alertId | UUID |
| patientId | UUID |
| ruleId | UUID |
| measurementId | UUID |
| status | AlertStatus |
| severity | Severity |
| createdAt | DateTime |
| acknowledgedAt | DateTime? |
| acknowledgedBy | UUID? |
| resolvedAt | DateTime? |

**Methods**

| |
|---|
| acknowledge(userId: UUID): void |
| resolve(): void |

**Class 6:** AIResult

AIResult stores the risk assessment output produced by the AI module in the patient context.

**Attributes**

| | |
|---|---|
| resultId | UUID |
| patientId | UUID |
| timeStamp | DateTime |
| riskScore | float |
| riskLevel | string |

**Methods**

| |
|---|

| |
|---|
| getFormattedResult(): string |
| |

**Class 7:** User

User represents a system account and stores the core identity fields required for role-based access control (RBAC).

**Attributes**

| | |
|---|---|
| userId | UUID |
| username | string |
| passwordHash | string |
| role | Role |
| isActive | boolean |
| createdAt | DateTime |
| displayName | string |

**Methods**

| |
|---|
| |
| verifyPassword(hash: string): boolean |
| |

### 3.1.2 Controller & Services

**Class 1:** DashboardController

DashboardController exposes API endpoints required by the frontend to retrieve patient details and active alerts.

**Methods**

| |
|---|
| |

| |
|---|
| getPatientDetails(patientId: UUID): PatientDTO |
| getActiveAlerts(patientId: UUID): List<AlertDTO> |
| acknowledgeAlert(alertId: UUID, userId: UUID): void |

**Class 2:** DataIngestionController

DataIngestionController receives incoming measurement data from the simulation/streaming layer and forwards it to the service layer for processing.

**Methods**

| |
|---|
| |
| receiveMeasurement(payload: VitalMeasurementDTO): void |

**Class 3:** AlertService

AlertService evaluates measurements against threshold rules, triggers alerts when needed, and manages acknowledge/resolve actions.

**Methods**

---

| |
|---|
| evaluateMeasurement(m: VitalMeasurement): void |
| triggerAlert(patientId: UUID, rule: ThresholdRule, m: VitalMeasurement): Alert |
| resolveAlert(alertId: UUID): void |
| acknowledgeAlert(alertId: UUID, userId: UUID): void |

**Class 4:** RuleEngineService

RuleEngineService retrieves patient-specific threshold rules and executes threshold checks for incoming measurements.

**Methods**

---

| |
|---|
| getRulesForPatient(patientId: UUID): List<ThresholdRule> |
| checkThresholds(patientId: UUID, m: VitalMeasurement): List<ThresholdRule> |

## 3.2 Frontend

### 3.2.1 View & Network

**Class 1:** DashboardPage

DashboardPage represents the dashboard UI layer and triggers chart rendering and alarm display actions.

**Attributes**

| | |
|---|---|
| activePatientId | UUID |

**Methods**

| |
|---|
| renderChart(): void |
| showAlarm(alert: AlertDTO): void |

**Class 2:** WebSocketClient

WebSocketClient manages the real-time WebSocket connection to the backend and forwards received messages to the UI layer.

**Attributes**

| | |
|---|---|
| connectionStatus | ConnectionStatus |

**Methods**

| |
|---|
| connect(): void |
| disconnect(): void |

| |
|---|
| onMessageReceived(msg: WebSocketMessage): void |
| subscribe(channel: String): void |

**Class 3:** ApiClient

ApiClient calls backend REST endpoints to perform standard data retrieval operations.

**Methods**

| |
|---|
| getPatientDetails(patientId: UUID): PatientDTO |
| getActiveAlerts(patientId: UUID): List<AlertDTO> |

### 3.3 AI Module

#### 3.3.1 Inference

**Class 1:** InferenceController

InferenceController is the external entry point for inference calls and returns an AIResult output.

**Methods**

| |
|---|
| infer(patientId: UUID, input: List<VitalMeasurementDTO>): AIResultDTO |

**Class 2:** InferenceService

InferenceService executes the inference flow, including preprocessing and model invocation.

**Methods**

| |
|---|
| runInference(patientId: UUID, input: List<VitalMeasurement>): AIResult |
| preprocess(input: List<VitalMeasurement>): FeatureVector |

**Class 3:** ModelRunner

ModelRunner performs prediction using the loaded model and outputs a risk score from a feature vector.

**Attributes**

| | |
|---|---|
| modelVersion | String |

**Methods**

| |
|---|
| predict(features: FeatureVector): float |
| |

### 3.4 Data Simulation

#### 3.4.1 Data Simulation

**Class 1:** ScenarioConfig

ScenarioConfig contains the parameter set required for synthetic data generation scenarios.

**Attributes**

| | |
|---|---|
| patientId | UUID |
| samplingRateHz | int |
| durationSeconds | int |
| signalTypes | List<SignalType> |

**Class 2:** Simulator

Simulator generates synthetic vital measurements.

**Methods**

| |
|---|
| generate(): VitalMeasurement |
| |

**Class 3:** StreamPublisher

StreamPublisher connects to the backend ingestion endpoint and publishes generated measurements.

**Methods**

| |
|---|
| connect(endpoint: String): void |
| publish(m: VitalMeasurement): void |
| start(config: ScenarioConfig): void |
| stop(): void |

### 3.5 Sequence Diagram

The following sequence diagram demonstrates the end-to-end interaction flow between system components during key operations including authentication, real-time vital monitoring, threshold evaluation, alert management, and AI inference.

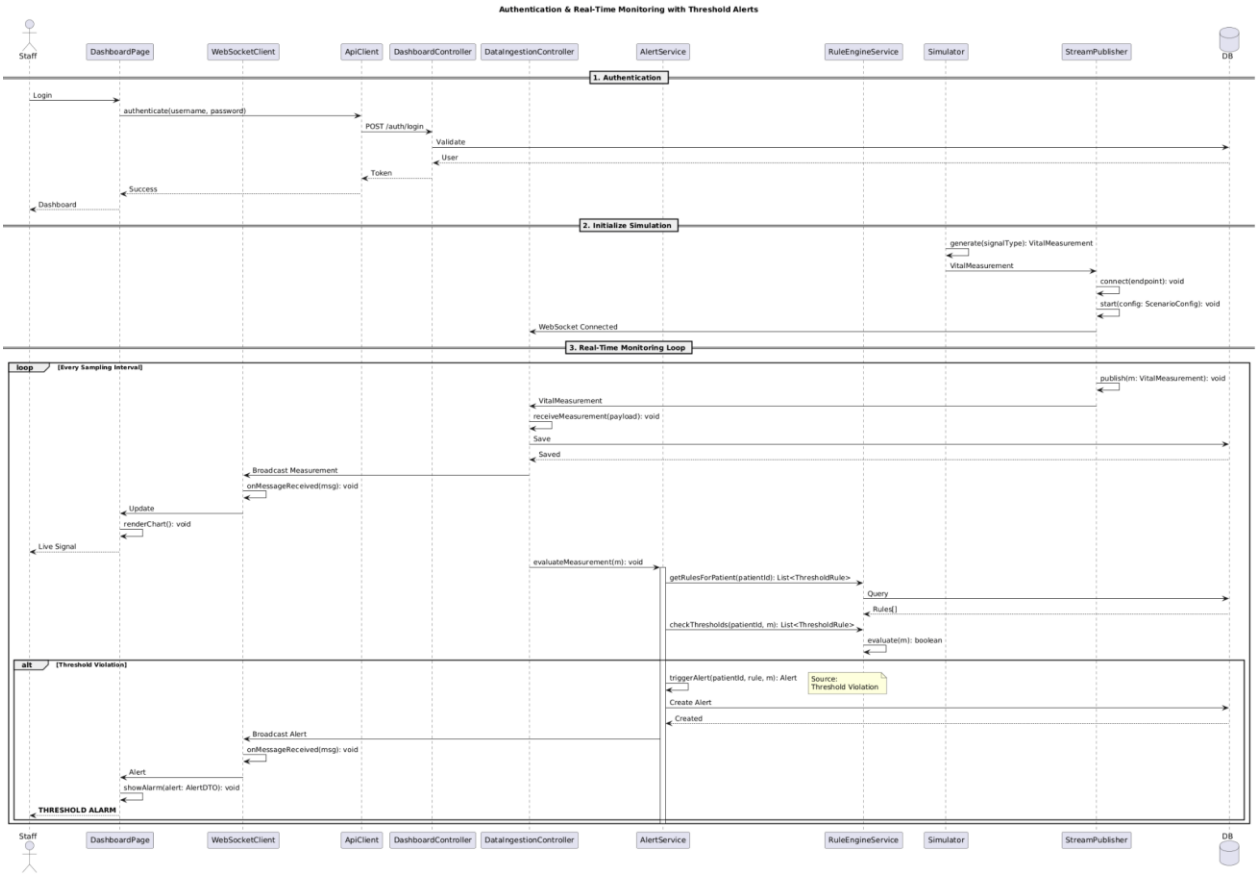


Figure 6: Authentication & Real-time Monitoring With Threshold Alerts Sequence Diagram

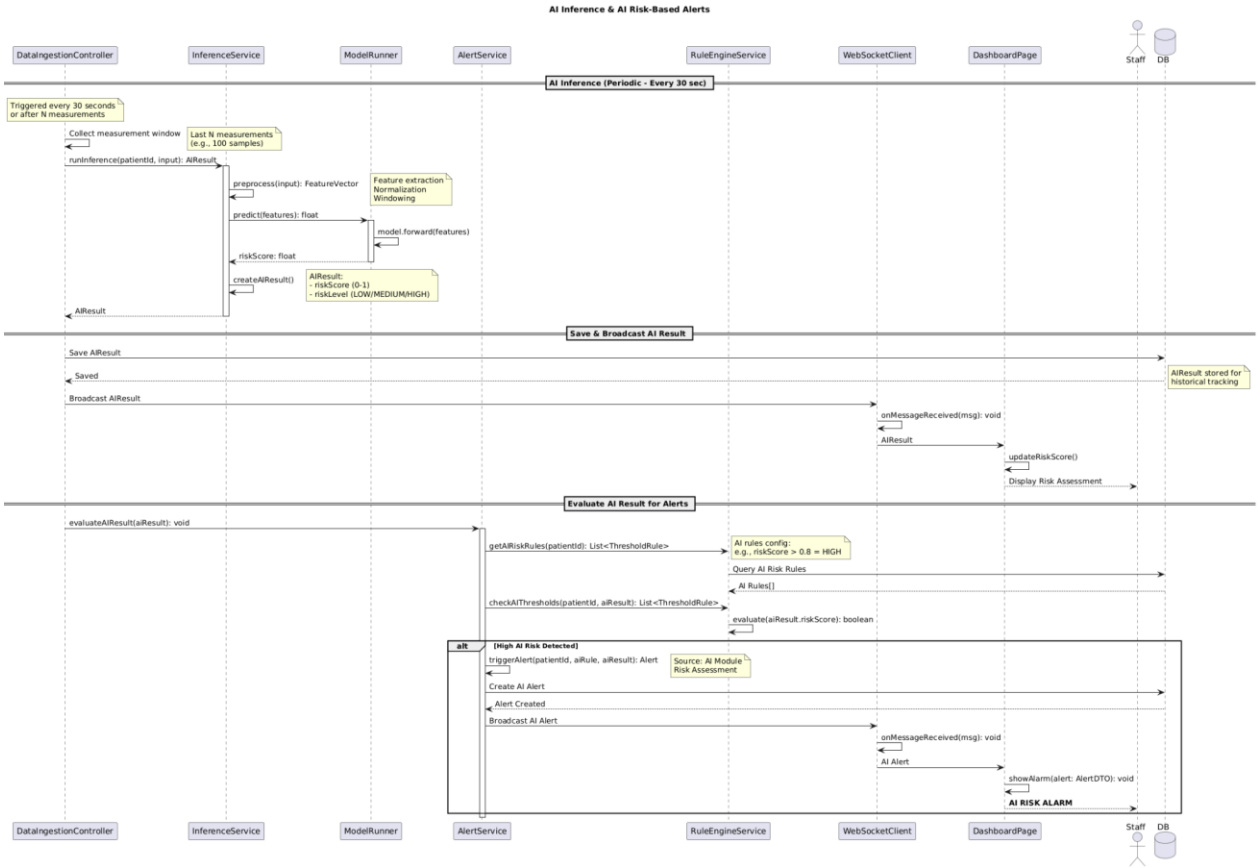


Figure 7: AI inference & AI Risk-Based Alerts Sequence Diagram

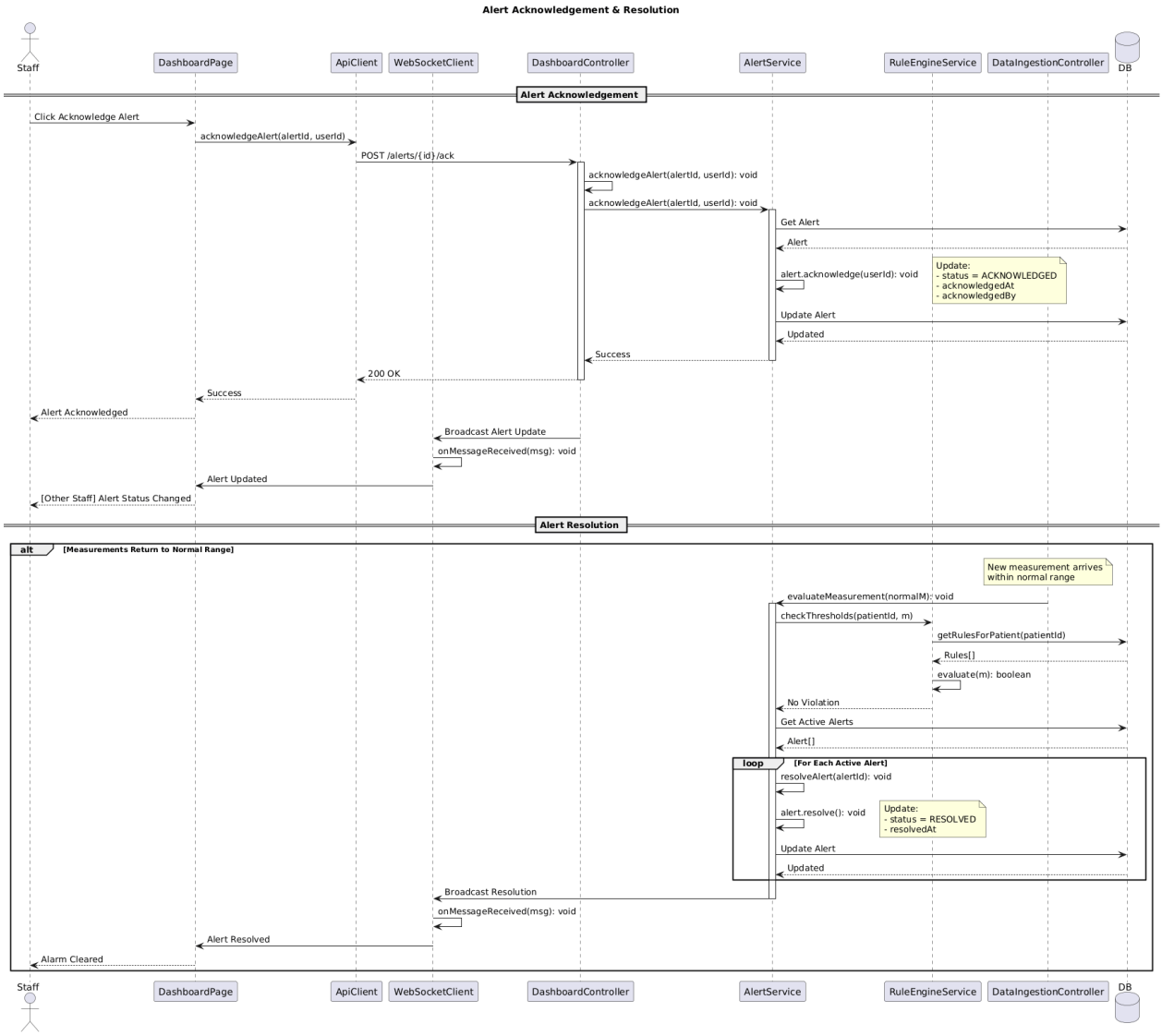


Figure 8: Alert Acknowledgement & Resolution Sequence Diagram

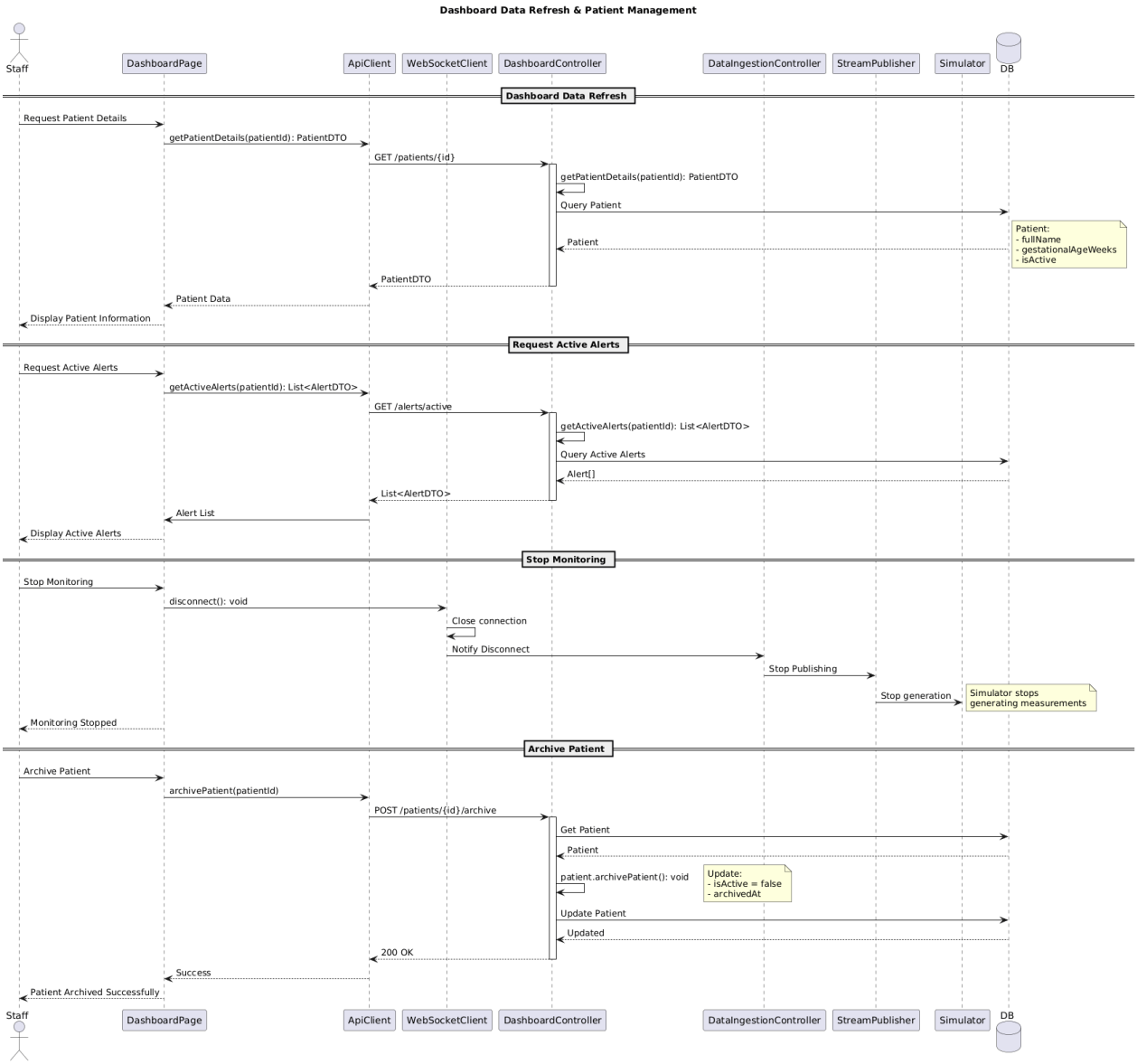


Figure 9: Dashboard Data Refresh & Patient Management Sequence Diagram

## 4. Glossary

This glossary provides concise explanations of key system components and technical terms used in this document. For acronyms and abbreviations, refer to Section 1.4.

### System Components

**Alert**

An alarm record triggered when a vital measurement violates threshold rules. Contains severity level and lifecycle status (ACTIVE, ACKNOWLEDGED, RESOLVED).

**AlertService**

Backend service that evaluates measurements against rules, creates alerts for violations, and manages acknowledgement/resolution.

**Backend Package**

Server-side component handling business logic, data storage, threshold evaluation, and API endpoints.

**Frontend Package**

Client-side dashboard interface for real-time monitoring, chart visualization, and alert management.

**InferenceService**

AI module service that preprocesses measurement data and generates risk predictions using machine learning models.

**Patient**

Domain entity representing a monitored neonate. Central context for all measurements, rules, alerts, and AI results.

**RuleEngineService**

Service that retrieves patient-specific threshold rules and checks measurements for violations.

**ThresholdRule**

Configuration defining acceptable minimum/maximum bounds for vital signs. Includes signal type, thresholds, and severity level.

**VitalMeasurement**

Single timestamped physiological measurement containing patient ID, signal type, value, and unit.

**WebSocketClient**

Frontend component managing real-time bidirectional communication with backend for live data streaming.

## 5. References


**Standards and Protocols**

**[1] HL7 FHIR** - Healthcare data interoperability standard (https://www.hl7.org/fhir/)

**[2] IEEE Std 1016-2009** - Software Design Descriptions standard

**[3] UML 2.5** - Unified Modeling Language for system diagrams (https://www.omg.org/spec/UML/)

**[4] WebSocket Protocol (RFC 6455)** - Real-time communication standard

**Security Standards**

**[5] TLS 1.3** - Transport layer security for encrypted communication

**[6] AES-256** - Encryption standard for data at rest

**[7] OWASP Top 10** - Web application security best practices

**Medical Device Standards**

**[8] IEC 60601-1-8** - Medical electrical equipment alarm systems

**[9] ISO 13485** - Medical devices quality management systems

**Architecture and Design**

**[10] Domain-Driven Design (Eric Evans)** - Domain model design principles

**[11] Clean Architecture (Robert C. Martin)** - Layered architecture patterns

**Project Documentation**

**Manifetch High-Level Design (HLD)** - Preceding architectural document

**Manifetch Project Specifications** - Functional and non-functional requirements